

git 버전 관리

git 저장소 만들기

[실습]

- ① hello-git 디렉터리 만들고 이동
- ② 디렉터리 내용 살펴보기 -> 파일도, 디렉터리도 없음
- ③ 깃 초기화
- ④ 다시 한번 디렉터리 내용 살펴보기 -> .git 디렉터리 생김

깃 초기화

```
$ git init
```

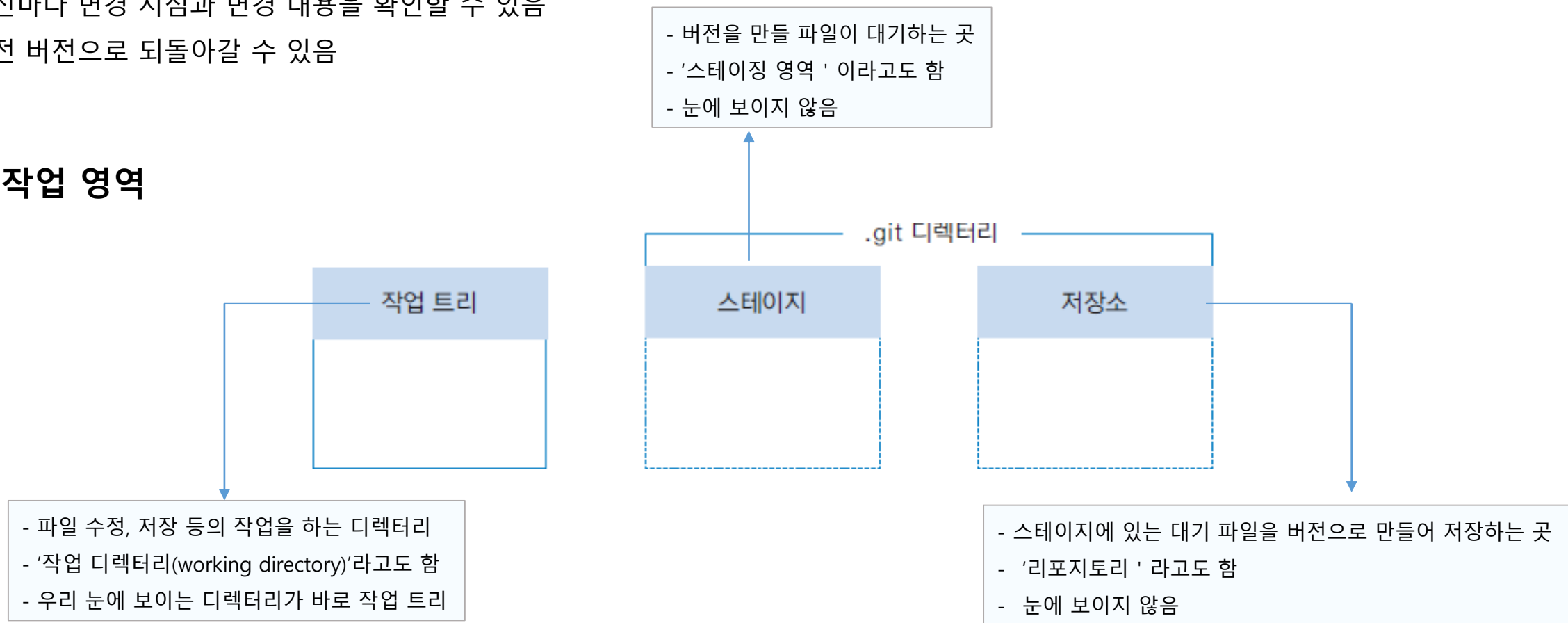
원하는 디렉터를 저장소로 사용할 수 있게 만들
초기화 후에 .git 디렉터리가 생김 (숨김 폴더)

버전 만들기

버전이란

- 깃에서 문서를 수정하고 저장할 때마다 생기는 것.
- 버전마다 변경 시점과 변경 내용을 확인할 수 있음
- 이전 버전으로 되돌아갈 수 있음

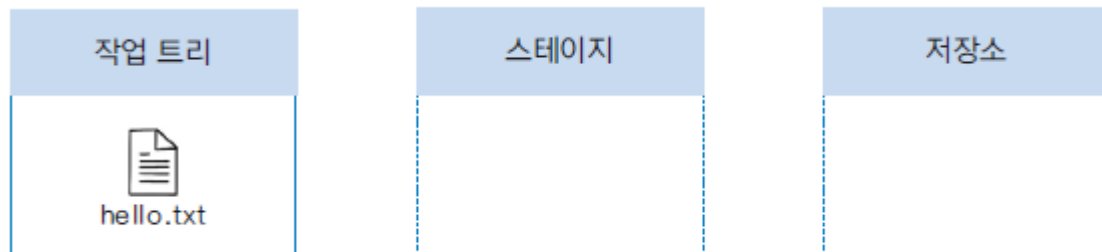
깃의 작업 영역



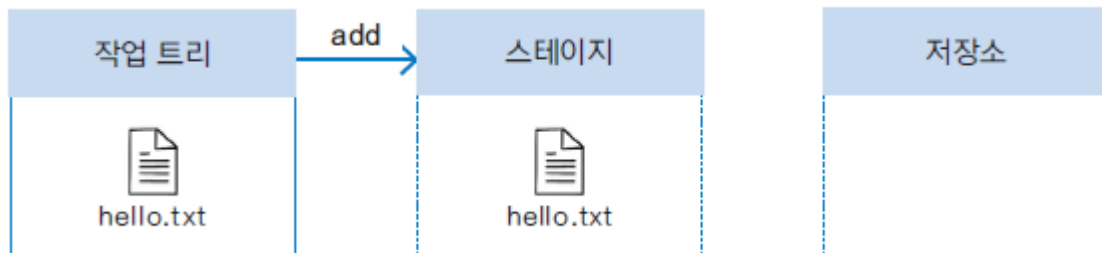
버전 만들기

스테이지와 커밋 이해하기

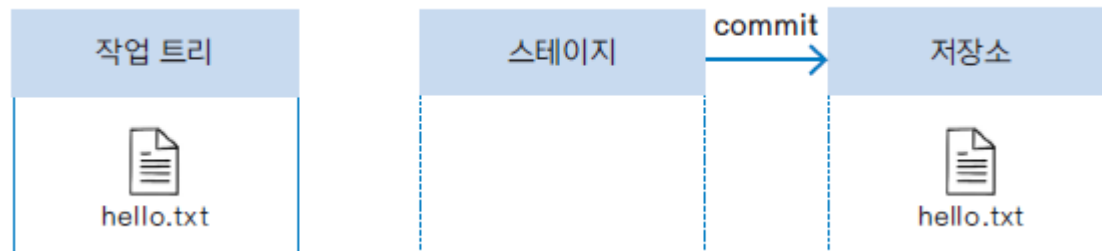
1) 작업 트리에서 파일을 수정하고 저장



2) 버전을 만들고 싶다면 스테이지에 넣음



3) 스테이지에 있던 파일을 저장소에 버전으로 저장



버전 만들기

[실습] VS Code에서 터미널 창 사용하기

- 1) [작업 폴더 추가] hello-git 폴더를 VS Code 위로 드래그하기
- 2) [터미널 창 열기] VS Code에서 [터미널] – [새 터미널] 선택, 또는, [Ctrl]+[Shift]+[`] 누름
- 3) 터미널 창이 열리면 'Git Bash'로 전환

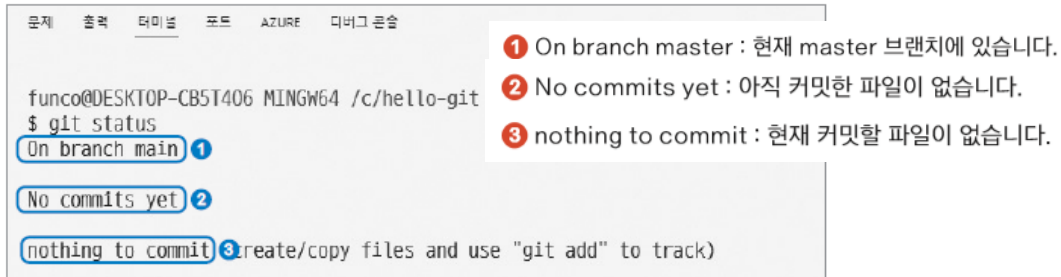


버전 만들기

[실습] 작업 트리에서 변경 사항 만들기

1) git status

저장소 상태 확인 (앞에서 초기화한 hello-git)



```
funco@DESKTOP-CB5T406 MINGW64 /c/hello-git
$ git status
On branch main
No commits yet
nothing to commit (create/copy files and use "git add" to track)
```

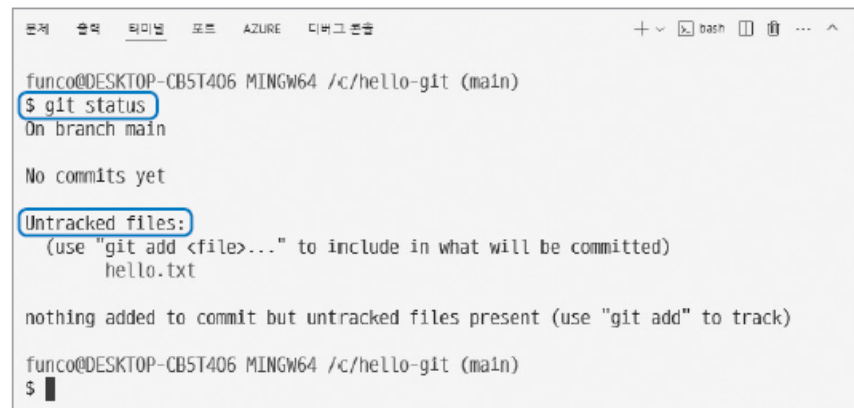
① On branch master : 현재 master 브랜치에 있습니다.
② No commits yet : 아직 커밋한 파일이 없습니다.
③ nothing to commit : 현재 커밋할 파일이 없습니다.

2) 새 파일 만들기

- ① VS Code에서 hello.txt 파일 만들고
- ② 숫자 '1' 입력
- ③ [Ctrl]+[S] 눌러 저장

3) git status

다시 저장소 상태 확인



```
funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    hello.txt

nothing added to commit but untracked files present (use "git add" to track)

funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$
```

버전 만들기

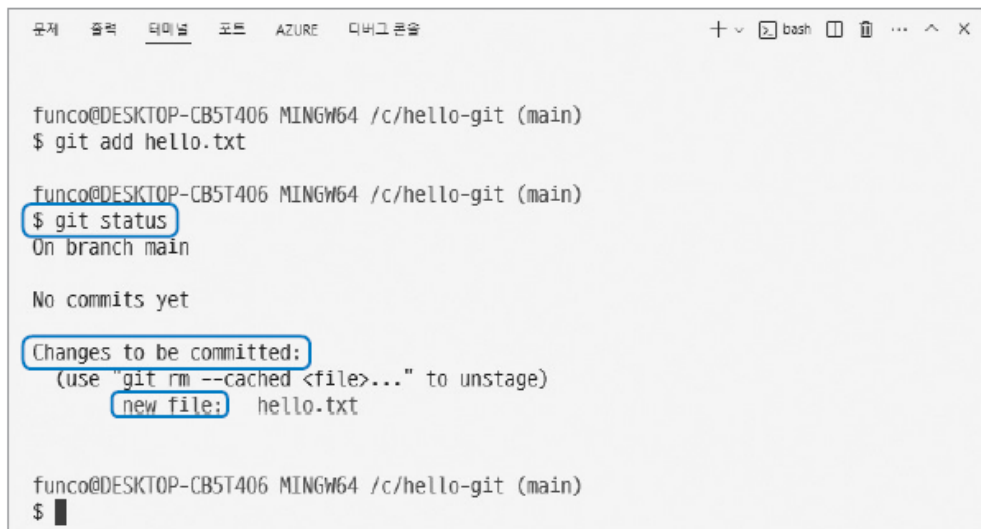
[실습] 스테이지에 변경 사항 올리기

1) **git add hello.txt**

작업 트리에서 수정한 파일(hello.txt)을 스테이지에 추가

2) **git status**

다시 저장소 상태 확인

A screenshot of a Windows terminal window with a title bar containing icons for file explorer, search, and other utilities. The terminal shows a series of git commands and their outputs. The first command is 'git add hello.txt'. The second command is 'git status', which shows the current branch as 'main' and lists 'hello.txt' as a new file to be committed. The terminal text is as follows:

```
funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$ git add hello.txt

funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   hello.txt

funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$
```

버전 만들기

[실습] 스테이징한 파일 커밋하기

1) **git commit -m "message1"**

- 버전을 만드는 것을 '커밋한다'라고 함
- 버전에 어떤 변경이 있었는지 메시지를 함께 기록해 둬.

```
funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$ git commit -m "message1"
[main (root-commit) 0f523b5] message1
1 file changed, 1 insertion(+)
create mode 100644 hello.txt
```

2) **git status**

다시 저장소 상태 확인

```
funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$ git status
On branch main
nothing to commit, working tree clean

funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$
```

3) **git log**

- 저장소에 있는 버전(커밋)을 확인
- 커밋을 만든 사람, 만든 시간, 커밋 메시지 등이 나타남

```
funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$ git log
commit 0f523b5495b06b54b86b4de91eebb5a2308904a2 (HEAD -> main)
Author: funbooki <funbooki@naver.com>
Date:   Fri Apr 19 14:53:14 2024 +0900

    message1

funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$
```

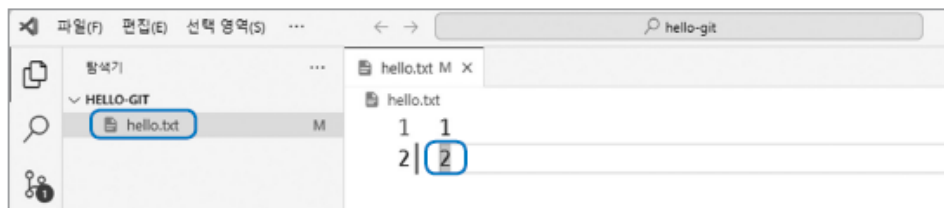
만든 사람 → Author: funbooki <funbooki@naver.com>
만든 시각 → Date: Fri Apr 19 14:53:14 2024 +0900
커밋 메시지 ← message1

버전 만들기

[실습] 스테이징과 커밋 한꺼번에 처리하기

1) **hello.txt** 수정

- 커밋하려면 수정 내용이 있어야 하므로
- 편집기에서 숫자 '2' 입력& 저장

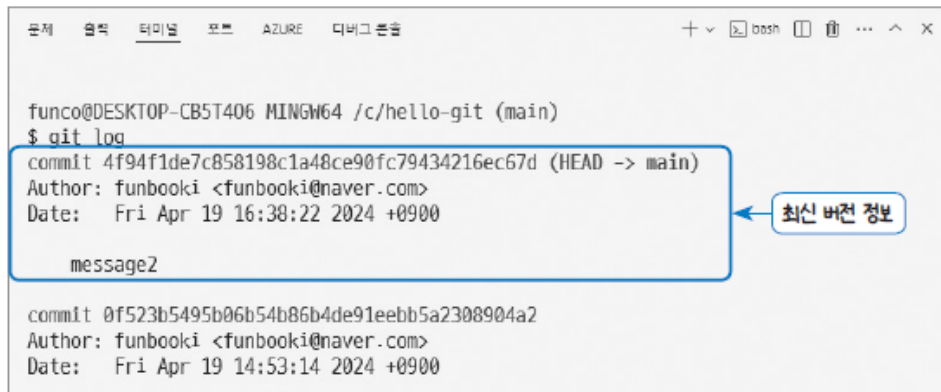


2) **git commit -am "message2"**

- commit 명령에 (add를 의미하는) -a 옵션 추가
- git commit -a -m 처럼 옵션을 따로 써도 됨.
- 스테이징과 커밋을 한꺼번에 처리하려면
무조건 한번은 커밋했던 파일이어야 함

3) **git log**

- 저장소에 있는 버전 확인
- 커밋을 만든 사람, 만든 시간, 커밋 메시지 등이 나타남



커밋 내용 확인하기

커밋 기록 자세히 살펴보기 - git log

- 커밋 기록 확인
- 지금까지 만든 버전이 나타나고 각 버전마다 설명이 함께 표시됨

```
funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$ git log
commit 4f94f1de7c858198c1a48ce90fc79434216ec67d (HEAD -> main)
Author: funbooki <funbooki@naver.com>
Date:   Fri Apr 19 16:38:22 2024 +0900

    message2

commit 0f523b5495b06b54b86b4de91eebb5a2308904a2
Author: funbooki <funbooki@naver.com>
Date:   Fri Apr 19 14:53:14 2024 +0900

    message1

funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$
```

커밋 내용 확인하기

변경 사항 확인하기 - git diff

특정 파일의 현재 상태와 최신 버전의 차이

1) hello.txt 수정

- 편집기에서 숫자 '2' 를 'two'로 변경



2) git status

- 아직 스테이징하지 않은 상태

3) git diff

- 수정한 파일과 저장소에 있는 파일 비교

'2' 삭제됨

'two' 추가됨

```
funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$ git diff
diff --git a/hello.txt b/hello.txt
index 7a754f4..669742f 100644
--- a/hello.txt
+++ b/hello.txt
@@ -1,2 +1,2 @@
 1
-2
\ No newline at end of file
+two
\ No newline at end of file

funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$
```

수정한 내용으로 다시 버전을 만들려면 스테이지에 올린 후 커밋하고
수정한 내용을 버리려면 git checkout 명령을 사용해 수정 내용 취소

버전 만드는 단계마다 파일 상태 확인하기

- 버전을 만드는 각 단계마다 파일의 상태가 달라짐.
- 파일의 상태를 이해하면 현재 어느 단계인지, 그 상태에서 어떤 일을 할 수 있는지 알 수 있음
 - tracked : 버전을 만든 후에 깃에서 변경 내역을 추적 중인 상태
 - untracked : 한번도 버전을 만들지 않은 상태
 - unmodified : 수정되지 않은 상태 (working tree is clean)
 - modified : 작업 트리에서 수정한 후 아직 스테이지에 저장하지 않은 상태
 - staged : 스테이지에 있고 아직 커밋하지 않은 상태

버전 만드는 단계마다 파일 상태 확인하기

[실습] tracked 파일과 untracked 파일

1) hello.txt 수정

-편집기에서 숫자 '3' 입력& 저장

2) hello2.txt 파일 만들기

- "a", "b", "c", "d" 한 줄씩 입력 & 저장

3) git status로 상태 확인

```
문제 출력 터미널 포트 AZURE 디버그 콘솔

funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   hello.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    hello2.txt

no changes added to commit (use "git add" and/or "git commit -a")

funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$ █
```

tracked 파일 :

한번이라도 커밋한 파일은 수정 사항 추적

untracked 파일 :

한번도 커밋하지 않은 파일. 수정 사항 추적안 함

버전 만드는 단계마다 파일 상태 확인하기

[실습] tracked 파일과 untracked 파일

4) git add .

- 마침표(.)를 사용하면 작업 트리의 파일을 한꺼번에 스테이징

5) git status로 상태 확인

```
funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   hello.txt
    new file:   hello2.txt

funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$
```

6) git commit -m "message3"

7) git log --stat

- 커밋과 관련 파일을 함께 확인

```
문제 출력 터미널 포트 AZURE 디버그 콘솔
+ ~ [git] [ ] [ ] ... ^ x

commit 87ad9b462dff54f43e6745c9bf92c846f4c83daa (HEAD -> main)
Author: funbook1 <funbook1@naver.com>
Date:   Fri Apr 19 16:49:23 2024 +0900

    message3

    hello.txt | 3 ++-
    hello2.txt | 4 +++
    2 files changed, 6 insertions(+), 1 deletion(-)

commit 4f94f1de7c858198c1a48ce90fc79434216ec67d
Author: funbook1 <funbook1@naver.com>
Date:   Fri Apr 19 16:38:22 2024 +0900

    message2

    hello.txt | 3 ++-
    1 file changed, 2 insertions(+), 1 deletion(-)

commit 0f523b5495b06b54b86b4de91eebb5a2308904a2
:
```

버전 만드는 단계마다 파일 상태 확인하기

[실습] unmodified, modified, staged 상태

1) hello-git 디렉터리에 있는 hello2.txt

2) git status로 상태 확인

```
문제 출력 터미널 포트 AZURE 디버그 콘솔

funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (ma:
$ git status
On branch main
nothing to commit, working tree clean

funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (ma:
$
```

unmodified 상태
변경 사항 없음

3) hello2.txt 수정

-편집기에서 'a'만 남기고 삭제 후 저장

staged 상태
커밋 직전 단계

4) git status로 상태 확인

modified 상태
수정 후 스테이지에 올리기 전

```
funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   hello2.txt

no changes added to commit (use "git add" and/or "git commit -a")

funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$
```

5) 스테이징 후 git status로 상태 확인

```
문제 출력 터미널 포트 AZURE 디버그 콘솔

funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$ git add hello2.txt

funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   hello2.txt

funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$
```

버전 만드는 단계마다 파일 상태 확인하기

[실습] unmodified, modified, staged 상태

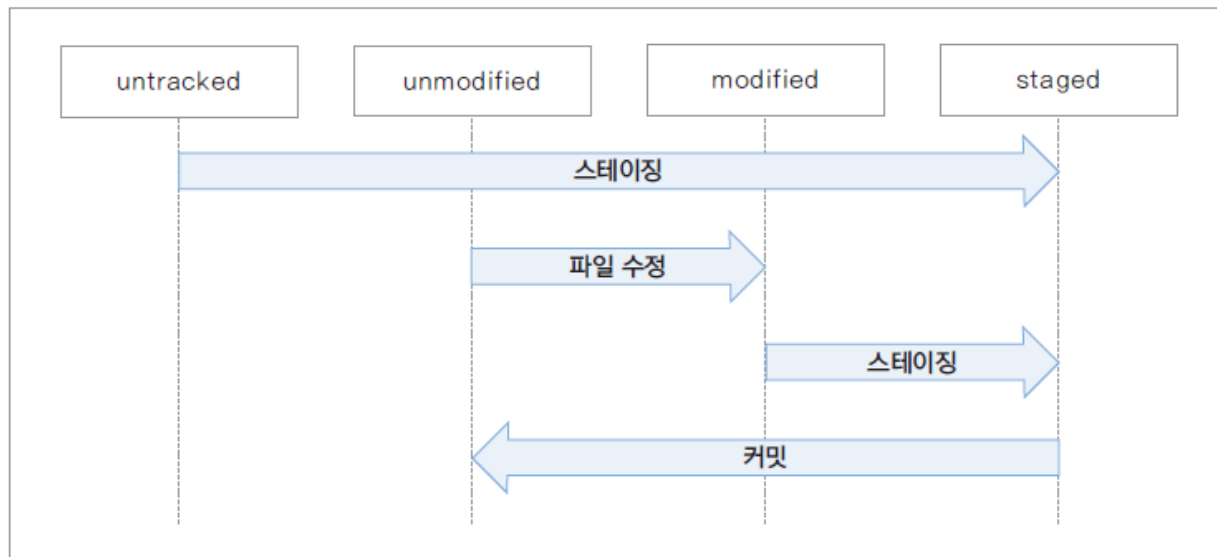
6) 커밋 후 git status로 상태 확인

```
문제 출력 터미널 포트 AZURE 디버그 콘솔

funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$ git commit -m "delete b, c, d"
[main 1fe03ff] delete b, c, d
1 file changed, 1 insertion(+), 4 deletions(-)

funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$ git status
On branch main
nothing to commit, working tree clean
```

unmodified 상태
변경 사항 없음



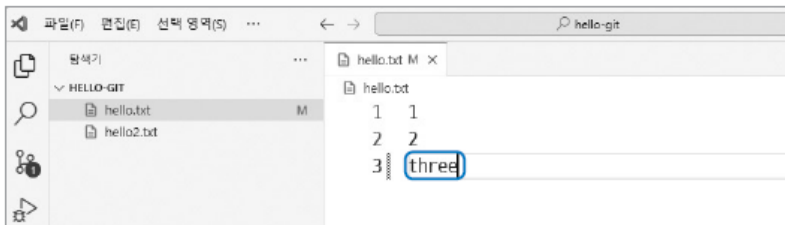
작업 되돌리기

작업 트리에서 수정한 파일 되돌리기

`git restore -- 파일명`

1) hello.txt 수정

-편집기에서 숫자 '3' 을 'three'로 수정 & 저장



2) git status로 상태 확인

```
funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   hello.txt

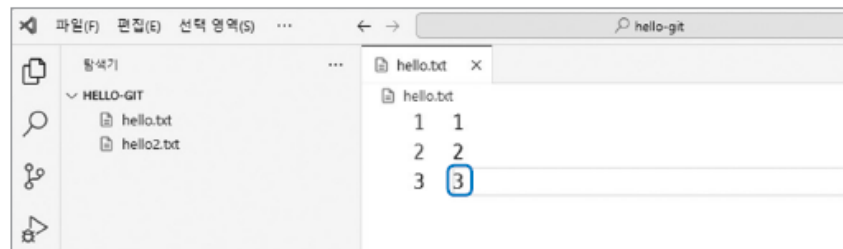
no changes added to commit (use "git add" and/or "git commit -a")

funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$
```

3) git restore 명령으로 수정 사항 취소

```
$ git restore hello.txt
```

4) hello.txt 파일 확인



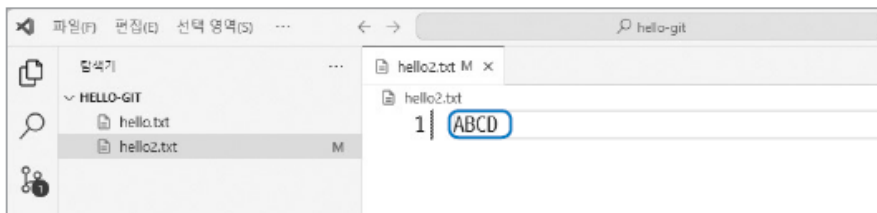
작업 되돌리기

스태이징 되돌리기

git restore --staged 파일명

1) hello2.txt 수정

- 기존 내용 삭제 후 'ABCD' 입력 & 저장



2) 스테이지에 올린 후 git status 상태 확인

```
funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$ git add hello2.txt

funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   hello2.txt

funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$
```

3) git restore 명령으로 스테이징 취소

```
$ git restore --staged hello2.txt
```

4) git status로 상태 확인

- 스테이지에 올라가기 전으로 돌아옴

```
funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$ git restore --staged hello2.txt

funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   hello2.txt

no changes added to commit (use "git add" and/or "git commit -a")

funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$
```

작업 되돌리기

최신 커밋 되돌리기

git reset HEAD^

1) hello2.txt 수정

- 기존 내용 끝에 'E' 추가 & 저장

2) git commit 명령으로 스테이징 & 커밋

```
$ git commit -am "message4"
```

3) git log 명령으로 커밋 확인

```
funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$ git log
commit 7262dab40740ef05dfa07ebeabd18b5e3ba6c09 (HEAD -> main)
Author: funbooki <funbooki@naver.com>
Date:   Fri Apr 19 22:05:25 2024 +0900

    message4

commit cb18017d6ecab9eb5f099f801030d1504db838e6
Author: funbooki <funbooki@naver.com>
Date:   Fri Apr 19 16:59:30 2024 +0900

    Modified - delete b, c, d

commit 87ad9b462dff54f43e6745c9bf92c846f4c83daa
Author: funbooki <funbooki@naver.com>
Date:   Fri Apr 19 16:49:23 2024 +0900
```

4) git reset 명령으로 최신 커밋 취소

- main의 최신 커밋도 취소되고 스테이지에서도 내려짐

```
$ git reset HEAD^
```

```
funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$ git reset HEAD^
Unstaged changes after reset:
M   hello2.txt

funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$
```

5) git log로 커밋 확인

```
funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$ git log
commit cb18017d6ecab9eb5f099f801030d1504db838e6 (HEAD -> main)
Author: funbooki <funbooki@naver.com>
Date:   Fri Apr 19 16:59:30 2024 +0900

    Modified - delete b, c, d

commit 87ad9b462dff54f43e6745c9bf92c846f4c83daa
Author: funbooki <funbooki@naver.com>
Date:   Fri Apr 19 16:49:23 2024 +0900

    message3
```

작업 되돌리기

특정 커밋으로 되돌리기

`git reset --hard 커밋해시`

1) 커밋 몇 개 만들기

- ① rev.txt 만들고 'a' 입력 저장 -> 스테이징 후 커밋(R1)
- ② 'b' 추가 후 저장 -> 스테이징 & 커밋(R2)
- ③ 'c' 추가 후 저장 -> 스테이징 & 커밋(R3)
- ④ 'd' 추가 후 저장 -> 스테이징 & 커밋(R4) 기존 내용 끝에 'E' 추가 & 저장

2) git log 명령으로 커밋 확인

```
funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$ git log
commit fa017bb72923e15780e5ac6d6453252803bc22d1 (HEAD -> main)
Author: funbooki <funbooki@naver.com>
Date: Fri Apr 19 23:20:10 2024 +0900
R4

commit abdd70fa4bb0848ed43cbb2d3abcce8af9f9e7f
Author: funbooki <funbooki@naver.com>
Date: Fri Apr 19 23:19:58 2024 +0900
R3

commit 9bf910267972964a0e092a781459f3bee0f2f796
Author: funbooki <funbooki@naver.com>
Date: Fri Apr 19 23:18:28 2024 +0900
R2

commit f45045ba6ef77797b2e390f9ac2dd02c4caf7e1a
Author: funbooki <funbooki@naver.com>
Date: Fri Apr 19 23:13:06 2024 +0900
R1
```

3) R2 커밋으로 되돌리기

- R3, R4 커밋 삭제, R2 커밋이 최신 커밋이 되도록
- HEAD를 R2로 리셋한다는 의미
- R2 커밋 해시 복사

```
Author: funbooki <funbooki@naver.com>
Date: Fri Apr 19 23:19:58 2024 +0900

R3

commit 9bf910267972964a0e092a781459f3bee0f2f796 ← 복사
Author: funbooki <funbooki@naver.com>
Date: Fri Apr 19 23:18:28 2024 +0900

R2

commit f45045ba6ef77797b2e390f9ac2dd02c4caf7e1a
Author: funbooki <funbooki@naver.com>
Date: Fri Apr 19 23:13:06 2024 +0900
```

작업 되돌리기

특정 커밋으로 되돌리기

`git reset --hard` 커밋해시

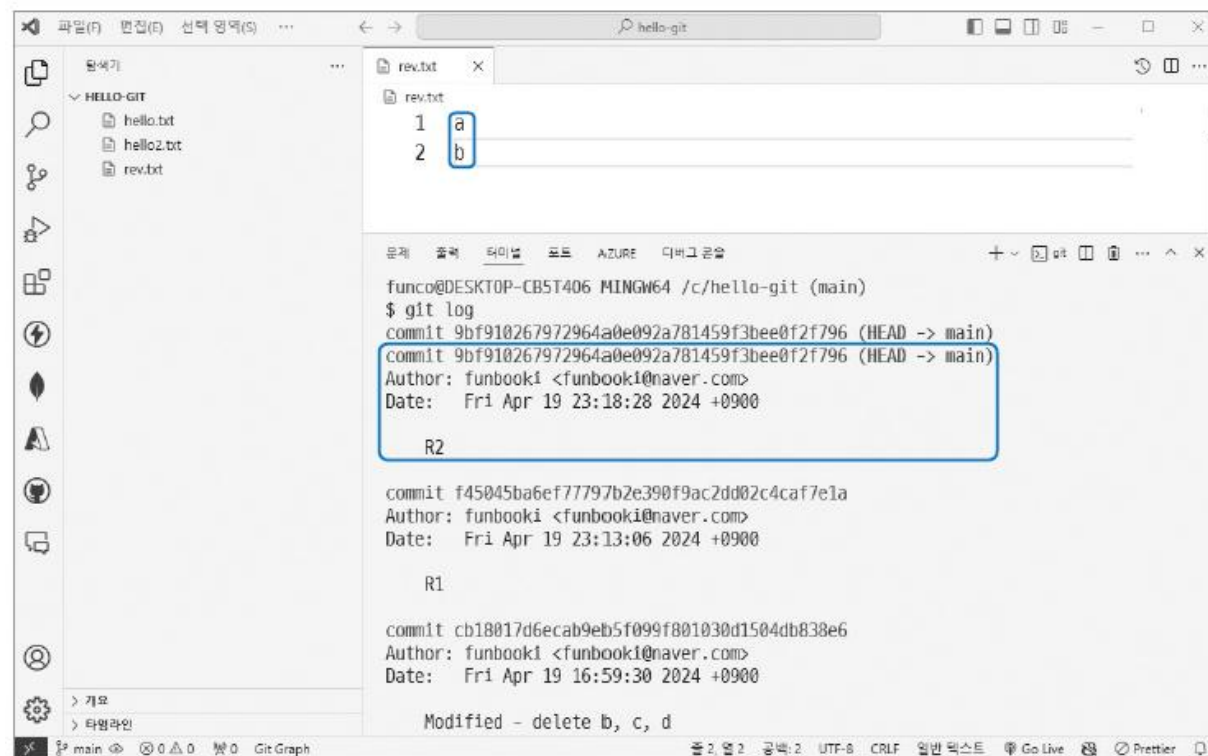
4) --hard 옵션과 함께 git reset으로 되돌리기

```
$ git reset --hard 복사한 커밋 해시
```

```
funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$ git reset --hard 9bf910267972964a0e092a781459f3bee0f2f796
HEAD is now at 9bf9102 R2

funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$
```

5) git log로 커밋 확인 & rev.txt 파일 확인



```
rev.txt
1 a
2 b

funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$ git log
commit 9bf910267972964a0e092a781459f3bee0f2f796 (HEAD -> main)
commit 9bf910267972964a0e092a781459f3bee0f2f796 (HEAD -> main)
Author: funbook1 <funbook1@naver.com>
Date: Fri Apr 19 23:18:28 2024 +0900

R2

commit f45045ba6ef77797b2e390f9ac2dd02c4caf7e1a
Author: funbook1 <funbook1@naver.com>
Date: Fri Apr 19 23:13:06 2024 +0900

R1

commit cb18017d6ecab9eb5f099f801030d1504db838e6
Author: funbook1 <funbook1@naver.com>
Date: Fri Apr 19 16:59:30 2024 +0900

Modified - delete b, c, d
```

작업 되돌리기

커밋 변경 이력 취소하기

git revert 커밋해시

- 커밋을 취소하면서도 커밋한 기록은 남겨 두어야 할 때

1) rev.txt 수정 & 커밋

- 기존 내용 끝에 'e' 추가 & 저장
- R5 메시지와 함께 커밋

2) git log 명령으로 커밋 확인

```
funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$ git log
commit 63239ca796d904286a5ceabcf7b59210ceb1a998 (HEAD -> main)
Author: funbook1 <funbook1@naver.com>
Date: Sat Apr 20 10:24:56 2024 +0900

R5

commit 9bf910267972964a0e092a781459f3bee0f2f796
Author: funbook1 <funbook1@naver.com>
Date: Fri Apr 19 23:18:28 2024 +0900

R2

commit f45045ba6ef77797b2e390f9ac2dd02c4caf7e1a
Author: funbook1 <funbook1@naver.com>
Date: Fri Apr 19 23:13:06 2024 +0900

R1
```

3) 커밋 취소하면서도 기록 남기기

- 최신 커밋 R5 버전을 취소 & R5는 삭제하지 않도록
- 취소할 R5 커밋 해시 복사

```
funco@DESKTOP-CB5T406 MINGW64 /c/hello-git (main)
$ git log
commit 63239ca796d904286a5ceabcf7b59210ceb1a998 (HEAD -> main)
Author: funbook1 <funbook1@naver.com>
Date: Sat Apr 20 10:24:56 2024 +0900

R5

commit 9bf910267972964a0e092a781459f3bee0f2f796
Author: funbook1 <funbook1@naver.com>
Date: Fri Apr 19 23:18:28 2024 +0900

R2
```

작업 되돌리기

커밋 변경 이력 취소하기

`git revert` 커밋해시

4) git revert 명령으로 취소하기

```
$ git revert 복사한 커밋 해시
```

5) 자동으로 커밋 메시지 창 표시됨

- 필요하면 수정 가능



6) git log로 커밋 확인

- R5 커밋을 revert한 커밋이 생김
- R5 커밋도 그대로지만, R5의 수정 내용은 취소됨

